

Sistemas Distribuidos

Docentes:

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Centralizar vs Distribuir

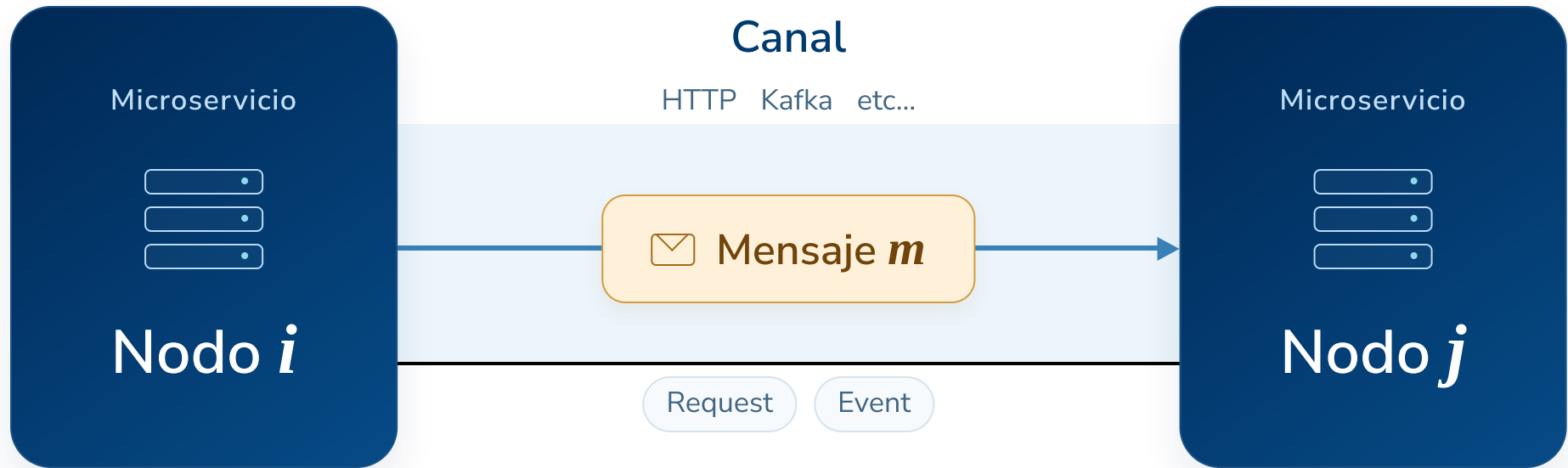
Centralizar tiene beneficios importantes que perdemos al distribuir:

- Un unico artefacto a desplegar.
- Debugging directo
- Transacciones ACID reales
- Latencia mínima
- Consistencia inmediata

No siempre distribuir es buena idea, perdemos estas ventajas pero ganamos otras

Criteria Básicos

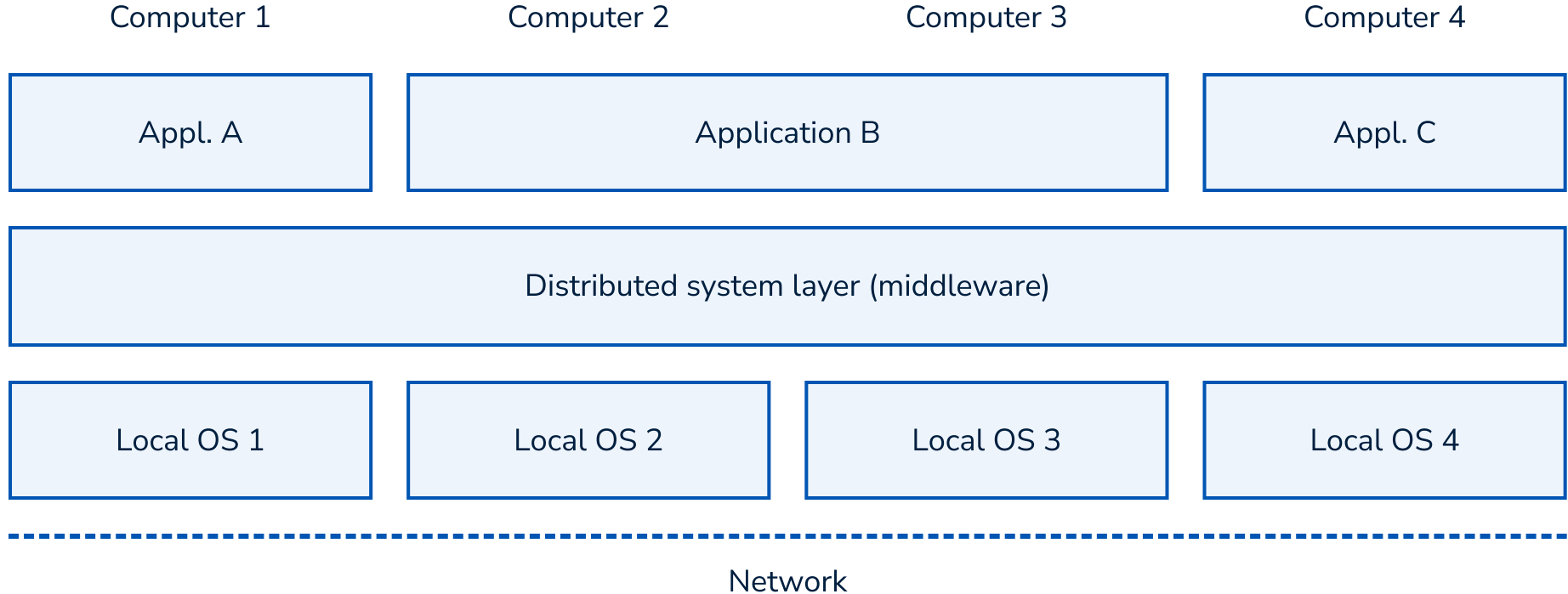
Modelo básico de sistema distribuido



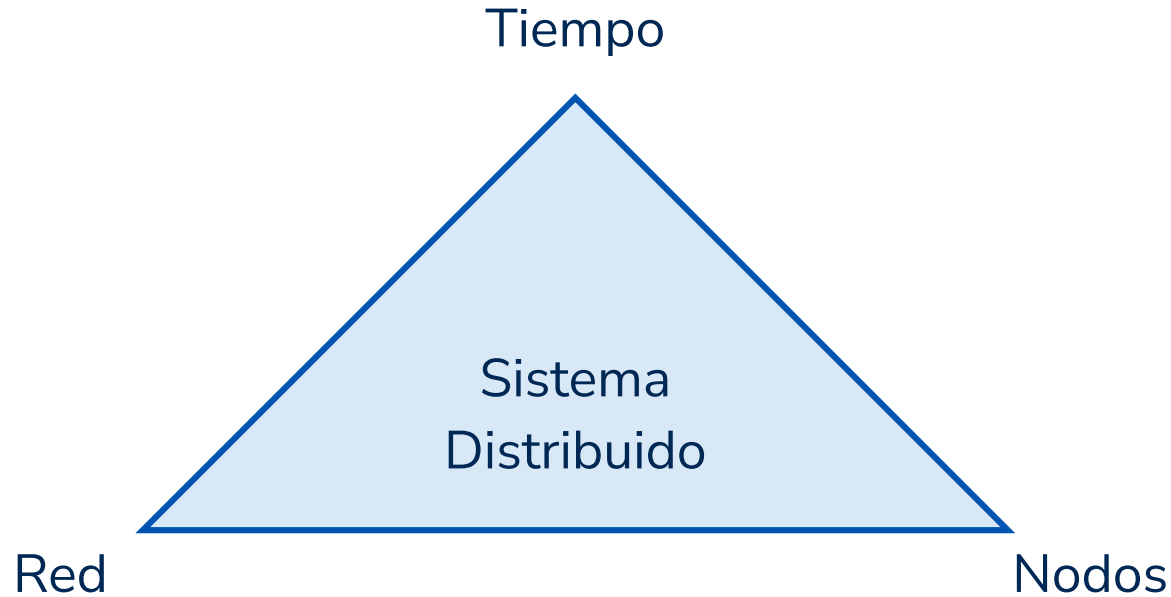
¿Por qué hacer un sistema distribuido?

- Aplicación inherentemente distribuida: mandar un mensaje de tu celular al de un amigo.
- Mejorar la confiabilidad: incluso si un nodo falla, el sistema como un todo se mantiene en funcionamiento.
- Mejorar la performance: por ejemplo, colocar el dato cerca del usuario en lugar de al otro lado del mundo.
- Resolver problemas más grandes: cantidad de datos que no caben en una sola máquina
- Tolerancia a Fallos: construir un sistema que tolere que algunas partes fallen.

Arquitectura tipo de un Sistema Distribuido



Comportamiento de Sistema Distribuido



Comportamientos de la Red

- Enlace confiable: un mensaje es recibido si fue enviado. Los mensajes pueden llegar reordenados.
- Fair-loss: los mensajes se pueden perder, duplicar o reordenar. No hay garantía de entrega.
- Enlaces arbitrarios: un adversario puede interferir con los mensajes: (hustear, modificar eliminar editar o replicar) Mitiga: TLS
- Demoras arbitrarias: un mensaje puede estar perdido o demorado.
- Partición de red: Dos grupos de nodos no pueden comunicarse entre sí, pero sí internamente.

Comportamiento de los nodos

- Crash-stop (fail-stop): El nodo está defectuoso software o hardware. Detiene su ejecución permanentemente.
- Crash-recovery (fail-recovery): El nodo crashea en cualquier momento, perdiendo su estado en memoria. Puede reasumir la ejecución en algún momento.
- Byzantine (fail-arbitrary): Un nodo defectuoso puede hacer cualquier cosa, crashear o comportarse de manera maliciosa.
- Un nodo no sabe si otro nodo está correcto o fallando.

Comportamiento del tiempo

- Synchronous: la latencia de mensajes no es mayor a un límite superior conocido. El algoritmo se ejecuta a una velocidad conocida.
- Asynchronous: Los mensajes pueden ser demorados de forma arbitraria. Los nodos pueden pausar su ejecución de forma arbitraria. No hay garantías de tiempo.

Variaciones de la sincronización en la práctica

- La pérdida de mensajes requiere reintentar.
- Congestión causa encolamiento.
- Reconfiguración de red.
- Los nodos usualmente ejecutan el código a una velocidad predecible con pausas ocasionales:
 - Acción del scheduler del sistema operativo.
 - Interrupciones del recolector de basura.
 - Page Faults, swap y thrashing.

Conclusion

- No hay reloj global confiable
- No hay memoria compartida
- La red falla (y miente)
- No podés distinguir:
 - nodo caído
 - nodo lento
- No existe “estado único global”

Implementaciones

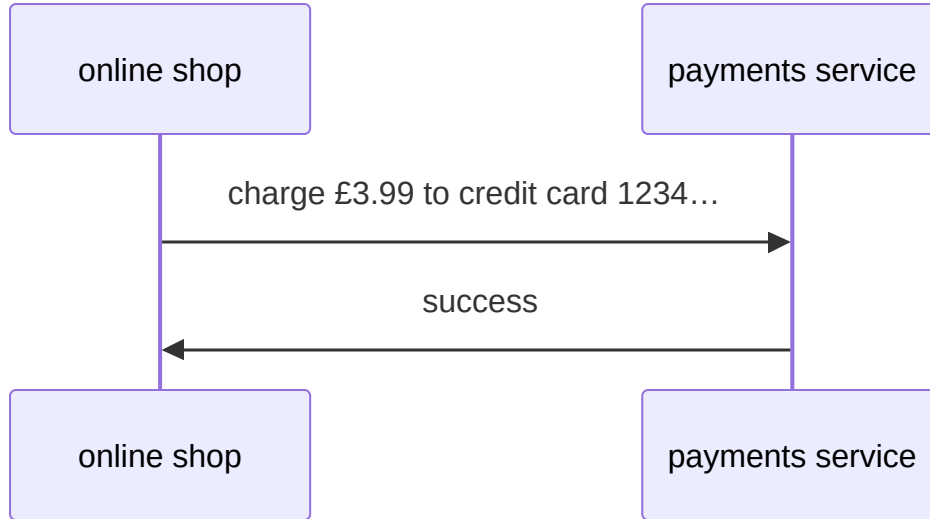
Remote Procedure Call

Remote Procedure Call

- El paradigma dominante para la comunicación síncrona entre servicios es el Remote Procedure Call o RPC.
- Hacer que una llamada a una función en otra máquina a través de la red se sienta exactamente igual que llamar a una función local en el mismo proceso.
- Transparencia de ubicación.

RPC

Client-server example: online payments



RPC

Remote Procedure Call (RPC) example

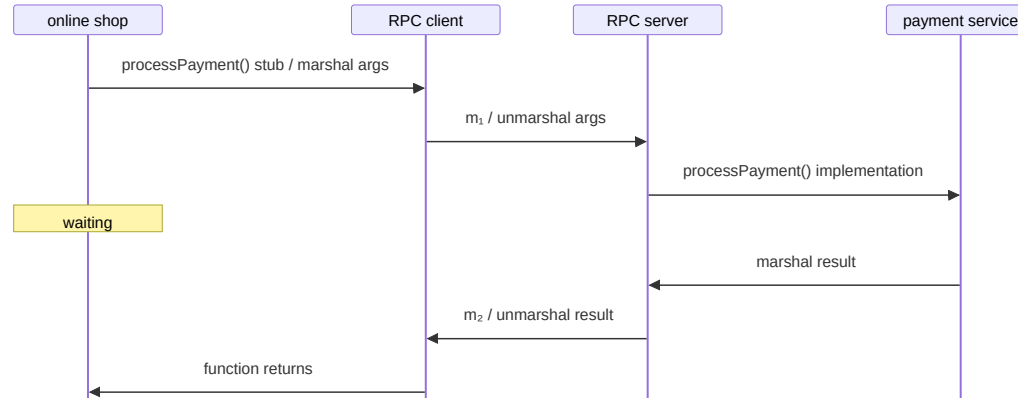
```
// Online shop handling customer's card details
Card card = new Card();
card.setCardNumber("1234 5678 8765 4321");
card.setExpiryDate("10/2024");
card.setCVC("123");

Result result = paymentsService.processPayment(card,
    3.99, Currency.GBP);

if (result.isSuccess()) {
    fulfilOrder();
}
```

Implementation of this function is on another node!

RPC



m_1

```
{
  "request": "processPayment",
  "card": { "number": "1234567887654321",
            "expiryDate": "10/2024", "CVC": "123" },
  "amount": 3.99, "currency": "GBP"
}
```

m_2

```
{
  "result": "success",
  "id": "XP61hHw2Rvo"
}
```

Problemas de RPC

Esta es la técnica mas basica para aprovechar la distribucion del sistema, sin embargo presenta problemas:

- Latencia variable
- Fallos parciales
- Timeout $\neq$ error seguro
- Reintentos $\rightarrow$ duplicación

Interfaz simple pero comportamiento complejo

Orquestación

El orquestador

- Puede ser un servicio dedicado o simplemente una parte del servicio existente.
- Define y controla el flujo completo.
- Envía comandos explícitos a cada servicio participante.
- Decide el siguiente paso en base a cada respuesta.
- Es el único componente que conoce la secuencia completa de principio a fin.
- Admite comunicacion sincrona o asincrona.

Atomic Commit

- Algunas operaciones necesitan modificar datos en múltiples nodos o servicios
- El desafío no es solo ejecutar la operación, sino garantizar que el resultado no quede parcial
- Atomic commit busca asegurar que todos los participantes confirmen la operación o que ninguno lo haga
- Problema:
 - un nodo confirma
 - otro falla
 - el sistema queda inconsistente

Aislamiento y serializabilidad

- Cuando varias transacciones se ejecutan al mismo tiempo, pueden interferirse entre sí
- Una ejecución es serializable si produce el mismo resultado que algún orden secuencial válido de esas transacciones
- Esto evita lecturas inconsistentes, pérdidas de actualización y otros errores de concurrencia

Transacciones Concurrentes

Escenario Inicial

- Estado inicial: $X = 10, Y = 10$
- Operación deseada: Transferir 1 dólar de X a Y y auditar el resultado
- Invariante del sistema: $X + Y = 20$ (la suma total debe mantenerse)

T1: TRANSFERENCIA

```
BEGIN
  add(X, 1)    // X = 11
  add(Y, -1)   // Y = 9
END
```

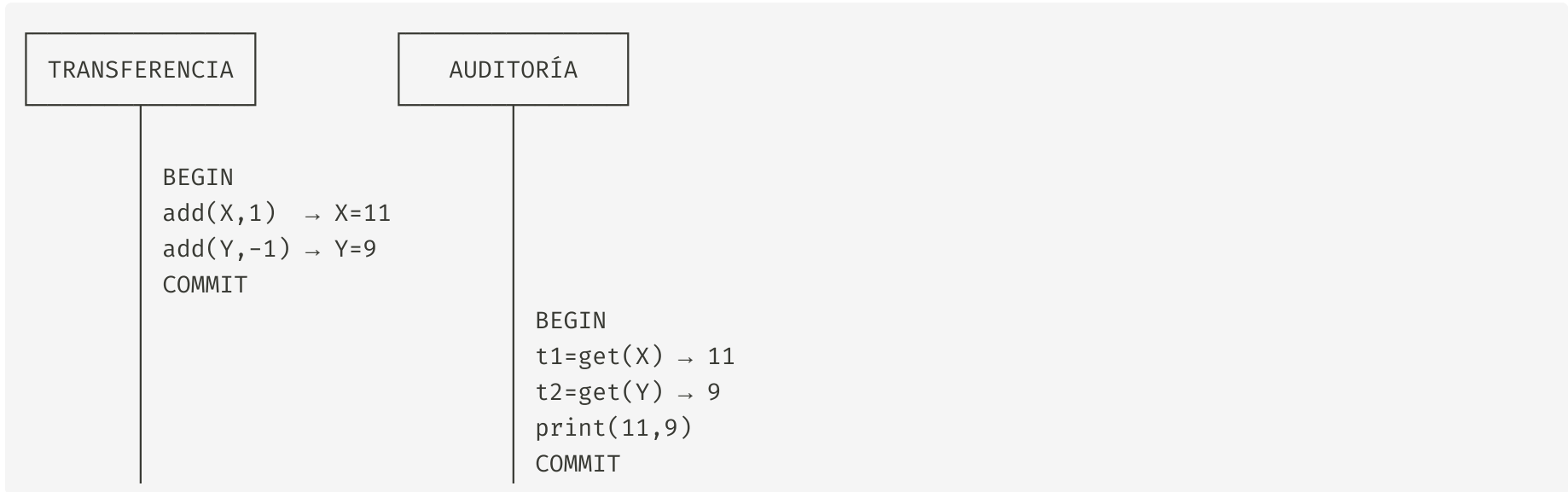
T2: AUDITORÍA

```
BEGIN
  t1 = get(X)
  t2 = get(Y)
  print(t1, t2)
END
```

Transacciones Concurrentes

Ejecuciones Posibles

Caso 1: T1 → T2



Resultado: X=11, Y=9, Auditoría muestra "11,9" ✓ (consistente)

Transacciones Concurrentes

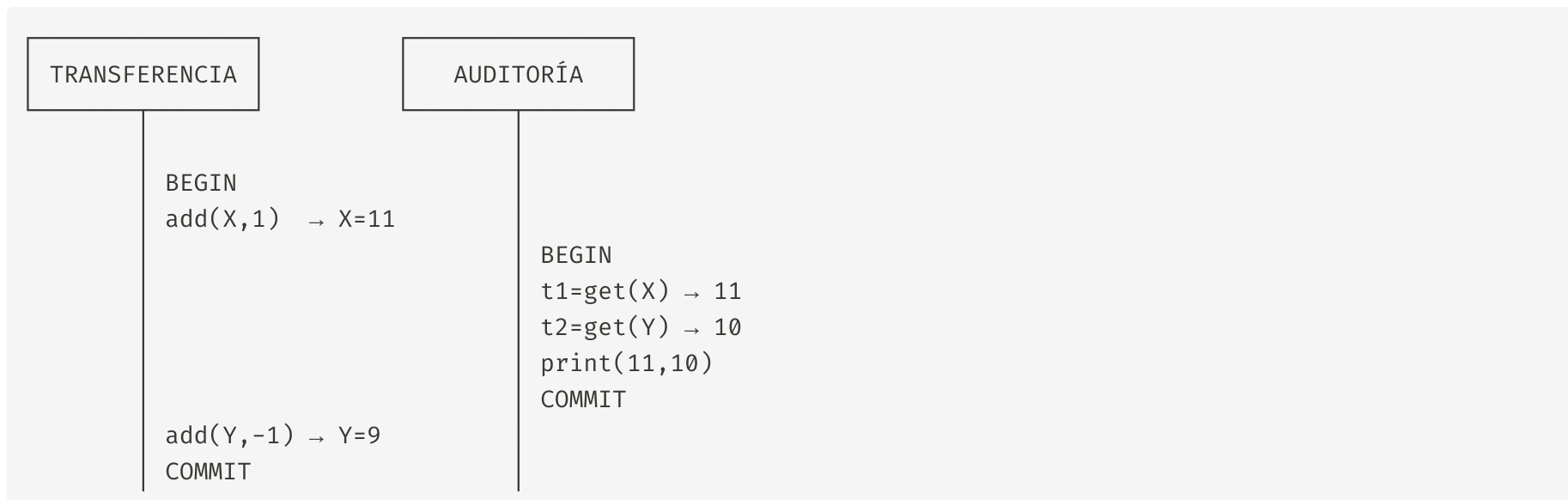
Caso 2: T2 → T1



Resultado: X=11, Y=9, Auditoría muestra "10,10" ✓ (consistente)

Transacciones Concurrentes

Caso 3: Ejecución Entrelazada (Problema de Consistencia)



Resultado: X=11, Y=9, Auditoría muestra "11,10" ✗ (inconsistente: suma = 21, se "creó" dinero)

Concurrency Control

	Control Pesimista	Control Optimista
Filosofía	“Prevenir antes que lamentar”	“Confiar y verificar”
Mecanismo	Bloquear antes de operar	Detectar conflictos al finalizar
Costo	Esperas y locks	Reintentos
Cuando Usar	Conflictos frecuentes	Conflictos poco frecuentes
Ejemplo	Reserva de asiento de avión	Edición de perfil de usuario

Optimistic Locking

- versionado (ej: version, updatedAt)
- CAS (compare and swap)
- retries automáticos

```
GET user (version=1)
UPDATE user (version=1 → 2)
Si alguien más lo cambió → conflicto → retry
```

Pesimistic: Two-Phase Commit

Permite coordinar una transaccion entre multiples nodos para lograr una real transaccion distribuida.

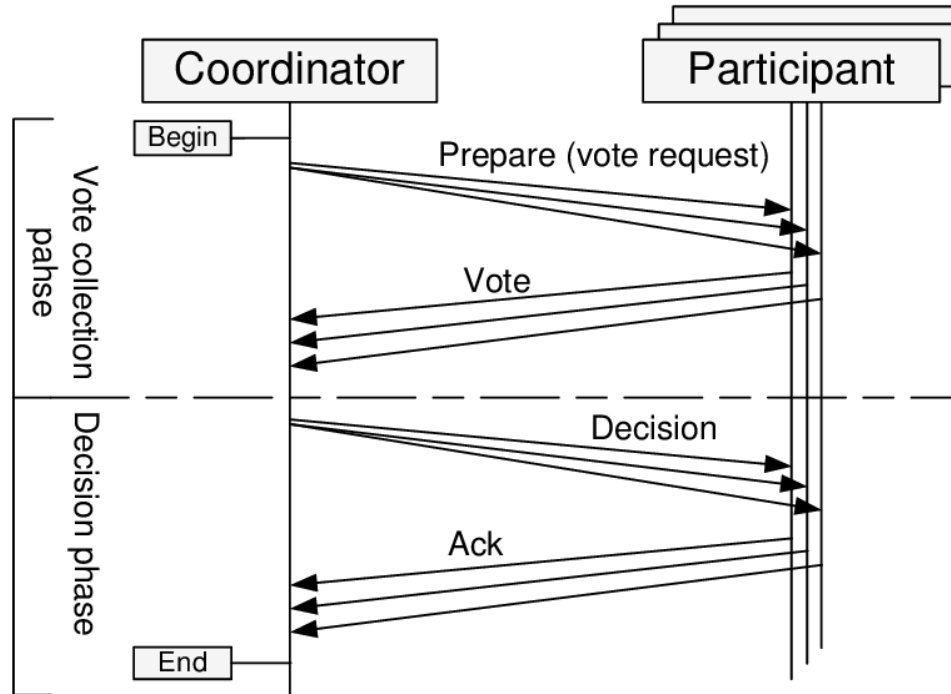
Todos los participantes confirman o todos abortan

- Fase de preparación: persiste la intencion y bloquea recursos.
- Fase de decisión: el coordinador decide ejecutar o no.
- Bloqueante
- Locks largos afectan concurrencia
- Single PoF
- Dificil escalabilidad

Two-Phase Commit

- Adquirir un lock antes de usar cualquier dato. Antes de leer o escribir cualquier récord.
- Una transacción debe acumular los locks hasta que finaliza, hasta que finaliza la transacción. Eso fuerza la serialización. T1 y T2 compiten para poder obtener el lock de x. La que primero obtiene ejecuta primero.
- Si las transacciones sueltan los locks en medio de su ejecución causa la no serialidad. Ej puede pasar que al final la transacción se aborte pero se leyó un dato equivocado.
- Es muy fácil que este tipo de locking produzca un deadlock

2 Phase commit



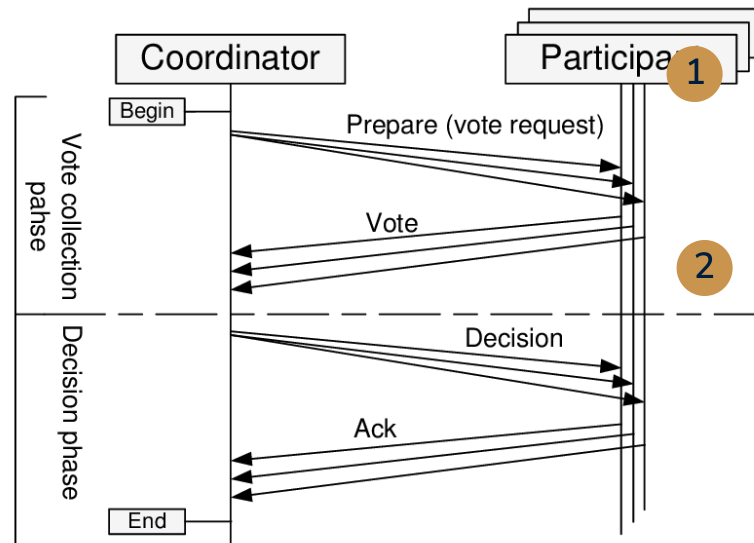
Escenarios de Crash (1)

1. Participante crashea antes de responder:

- TC nunca puede commitear la transacción

2. Participante crashea después de responder Sí:

- Se comprometió pero no puede ejecutar
- Otros servidores sí harán su parte
- Decisión de Sí del participante debe ser durable



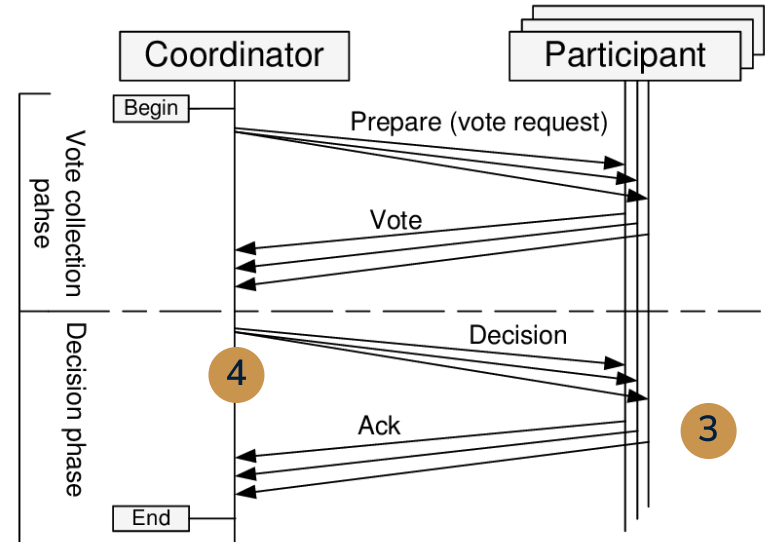
Escenarios de Crash (1)

3. Recepción múltiple de commit provenientes del coordinador:

- Simplemente envía nuevamente el ACK

4. TC crashea:

- Antes de enviar commits: se aborta
- Después de algunos commits: debe recordar la transacción (durable)



Escenarios de Crash (3)

- 5. TC puede abortar unilateralmente dentro de un timeout.
- 6. Participantes no pueden abortar o commitear unilateralmente:
 - Deben mantener el lock indefinidamente

Cuando usar two phase commit

- Necesitas consistencia inmediata
- No puede haber estados intermedios
- No puede haber compensaciones

Ejemplos:

- Sistemas financieros core (no puedes crear ni destruir dinero)
- Transacciones entre bases de datos
- Operaciones legales/ regulatorias
- Sistemas Legacy

Coreografía

Coreografia

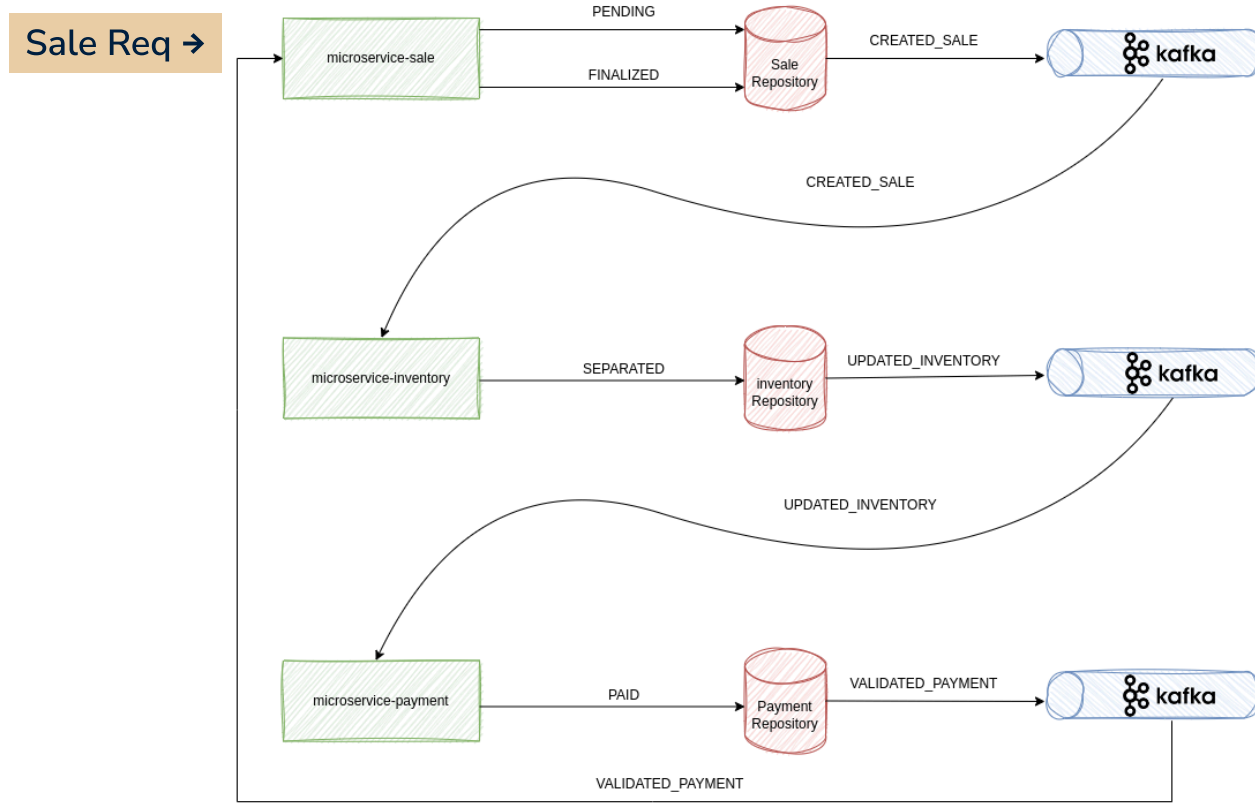
No existe orquestador central.

Cada microservicio reacciona a eventos o invoca a otros segun su rol.

El flujo de negocio esta implicito entre los servicios. Cada servicio conoce solo su parte.

Puede ser sincrona o asincrona (conmente basada en eventos)

- No hay punto unico de fallo
- Altamente escalable.
- Bajo acoplamiento.
- Cada servicio evoluciona independientemente
- Flujo dificil de entender.
- Debuggin complejo (sin owners).
- Requiere observabilidad.
- Dificil de mantener





Fin!!